A087

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA 3.18)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)
EXPERIMENT NO. 6

Name : Devyani .S. Kamat SAP : 60017240102

AIM:

To write operational schemas for an e-commerce catalog tracking system connecting Next.js with MongoDB through a Mongoose modeling structure.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Database Engine:** MongoDB (Local instance, Docker Container, or MongoDB Atlas)
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Document-Oriented Data Modeling with MongoDB:** MongoDB stores data as flexible, JSON-like BSON (Binary JSON) documents. Unlike rigid relational database management systems (RDBMS), MongoDB allows semi-structured schemas, polymorphic documents, and nested sub-documents, which are ideal for dynamic e-commerce product catalogs where attributes vary across categories.
- **Schema Abstraction with Mongoose:** Mongoose provides an Object Data Modeling (ODM) layer on top of the MongoDB native driver. It enforces strict application-level schema definitions, automatic type casting, field validation rules, default values, pre/post middleware lifecycle hooks, and virtual population across collections using Schema.Types.ObjectId.
- **Database Connection Caching in Next.js Serverless Environment:** Next.js Server Components and Server Actions execute in a stateless, ephemeral serverless/HMR environment. Calling mongoose.connect() inside standard route handlers or Server Actions without a connection caching mechanism creates a brand-new connection on every hot-reload or request, quickly exhausting the MongoDB connection pool. Caching the active connection promise on Node.js globalThis guarantees that server functions reuse existing connection handles across hot-module reloads.
- **Embedded Documents vs. Document Referencing:**
    - **Embedded (Denormalized) Sub-documents:** Used for attributes that are always retrieved together with the product, such as metadata, technical specifications, and inventory tracking dimensions.
    - **Referenced (Normalized) Documents:** Used for independent domain entities like Category or Vendor, mapped via ref pointers and populated using Mongoose .populate().

```
        </section>

      </div>
    </div>
  );
}
```

Step 6: Validate Catalog Workflow

1. Execute the Next.js development server:

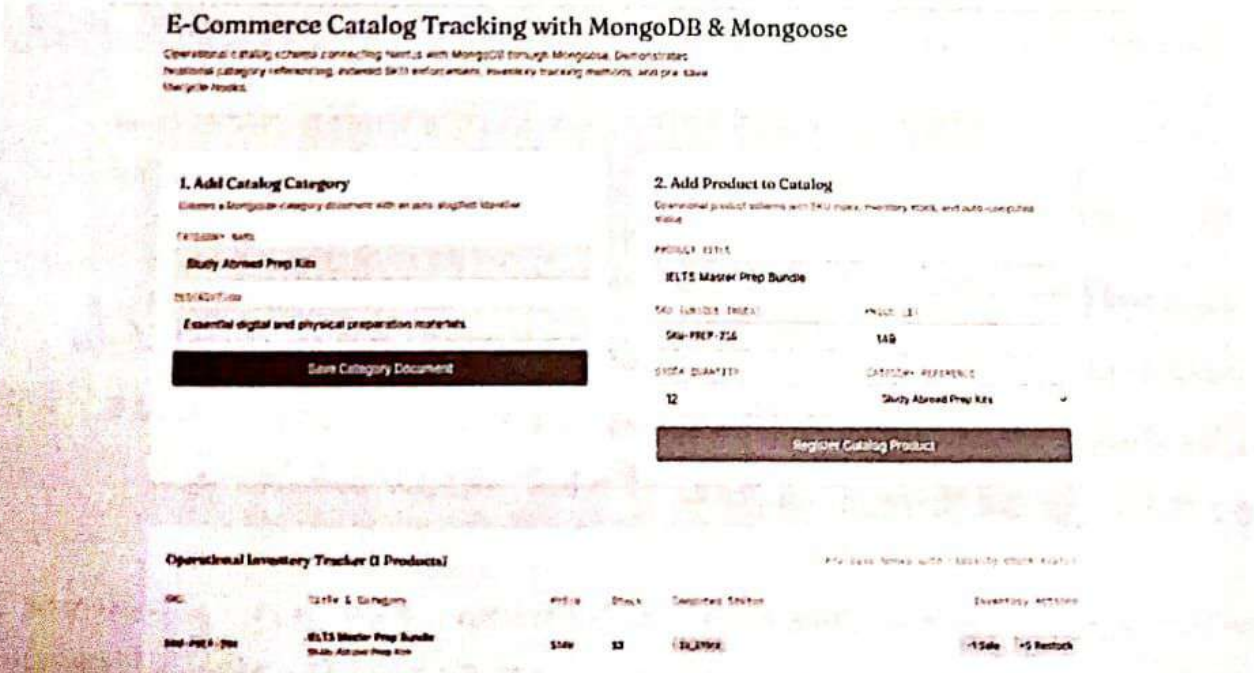
```
npm run dev
```

2. Navigate to http://localhost:3000/catalog in your browser.
3. Submit a new catalog item. Inspect the rendered cards to verify that Mongoose computed the slug, validated the category enum, embedded the inventory sub-document, and ran the pre-save hook to evaluate inStock.

OBSERVATION:

Observe how Mongoose enforces document validation rules prior to writing to the MongoDB database. If a non-whitelisted category or negative price is supplied, Mongoose validation halts execution and returns a structured validation error.

Note how embedded schemas (InventorySchema) are stored inline within the parent product document, allowing inventory levels and location coordinates to be read in a single round-trip query without relational joins.





CONCLUSION:

Operational schemas for an e-commerce catalog tracking system were successfully constructed using Next.js and MongoDB via Mongoose. The experiment verified singleton database connection caching, embedded sub-document schema modeling, pre-save lifecycle hooks, and server-side data mutations using React Server Actions.

POST LAB QUESTION:

Q: Why is global connection caching (globalThis.mongooseCache) mandatory when connecting Mongoose with Next.js App Router applications, and how does it differ from traditional Express.js database connection lifecycles?

A:

In traditional Express.js architectures, the Node.js server process runs continuously as a long-lived stateful instance where mongoose.connect() is called once at application startup.

In Next.js App Router (especially during development with Hot Module Replacement and in serverless production runtimes), server-side modules are re-evaluated frequently. Without global connection caching, each file modification or server request invokes mongoose.connect(), creating multiple unmanaged database connection sockets.

Caching the connection promise on globalThis guarantees that across hot-reloads and concurrent server operations, Mongoose reuses the existing open connection pool instead of exhausting the database connection limits.

Observation :
Verified singleton database connection caching, embedded sub-document schema modeling, pre-save lifecycle hooks, & server-side data mutations.

Conclusion : Operational schemas for an e-commerce catalog tracking system connecting Next.js with MongoDB through a Mongoose modelling structure was implemented.